Machine Learning Session: Ensemble Methods

Random Forest and XGBoost using caret in R

LUMUMBA W. VICTOR

Center for Data Analytics and Modeling
Chuka University

March 2026

Resource: https://cdam.chuka.ac.ke

Learning Objectives

By the end of this session, you will be able to:

1. Prepare health data with multi-level categorical variables
2. Understand ensemble learning principles (bagging, boosting)
3. Implement Random Forest using caret::train()
4. Implement XGBoost using caret::train()
5. Tune hyperparameters for both models
6. Interpret feature importance from ensemble models
7. Compare ensemble models with baseline methods

Dataset: Diabetes.csv (Health Indicators)
Target: diabetes (Yes/No)

Session Outline

Part A: Data Prep (Slides 4-10)

1. Loading and Cleaning
2. Categorical Variable Handling
3. smoking_history Categories
4. Train-Test Split
5. Preprocessing Setup

Part B: Random Forest (Slides 11-30)

6. Ensemble Learning Overview
7. Bagging Theory
8. Random Forest Algorithm
9. Hyperparameters
10. Implementation with caret
11. Tuning and Evaluation

Part C: XGBoost (Slides 31-50)

12. Boosting Theory
13. Gradient Boosting
14. XGBoost Advantages
15. Implementation with caret
16. Tuning and Evaluation

Part D: Comparison (Slides 51-55)

17. Model Comparison
18. Recommendations

Required Libraries

```r
# Core packages
library(caret)          # Unified ML interface
library(tidyverse)      # Data manipulation
library(pROC)           # ROC analysis
library(xgboost)        # XGBoost engine
library(randomForest)   # Random Forest engine
library(e1071)          # For SVM comparison

# Theme settings
theme_set(theme_minimal())

# Set seed for reproducibility
set.seed(2026)
```

Note: Install packages if needed: `install.packages()`

Loading the Data

```r
# Read the dataset
diabetes_data <- read_csv("Diabetes.csv")

# Inspect structure
glimpse(diabetes_data)
summary(diabetes_data)

# Check dimensions
dim(diabetes_data)  # rows x columns

# Check for missing values
colSums(is.na(diabetes_data))

# Variable names
names(diabetes_data)
```

Variables:

- age, hypertension, heart_disease
- smoking_history (6 categories)
- bmi, HbA1c_level, blood_glucose_level
- diabetes (target)

Categorical Variable Handling

Important Note: smoking_history has 6 categories, not binary!

```r
# Check smoking_history distribution
table(diabetes_data$smoking_history,
      useNA = "ifany")

# Convert to factor with proper levels
diabetes_data <- diabetes_data %>%
  mutate(
    # Target variable
    diabetes = factor(diabetes,
                      levels = c(0, 1),
                      labels = c("No", "Yes")),
```

Categorical Variable Handling

```r
    # Multi-level categorical: smoking_history
    smoking_history = factor(smoking_history,
                             levels = c("never", "ever",
                                        "former", "current",
                                        "not current", "No Info"),
                             ordered = FALSE),
    # Binary variables
    hypertension = factor(hypertension,
                          levels = c(0, 1),
                          labels = c("No", "Yes")),
    heart_disease = factor(heart_disease,
                           levels = c(0, 1),
                           labels = c("No", "Yes"))
  )

# Verify conversions
str(diabetes_data$smoking_history)
levels(diabetes_data$smoking_history)
```

caret automatically creates dummy variables for factors

smoking_history Distribution

Variable Summary (N = 100,000):

Category	N	Percent	Cumulative
never	35,095	35.10%	35.10%
No Info	35,816	35.82%	70.92%
former	9,352	9.35%	80.27%
current	9,286	9.29%	89.56%
not current	6,447	6.45%	96.01%
ever	4,004	4.00%	100.00%

Implications for Modeling:

- 6 dummy variables will be created (reference: "never")
- "No Info" is a valid category (missingness may be informative)
- Ensure adequate representation in train/test splits

Always check category distributions before modeling

Missing Values

```r
# Check missing values per variable
missing_summary <- diabetes_data %>%
  summarise(across(everything(),
                   ~sum(is.na(.))))
print(missing_summary)

# Option 1: Remove rows with missing values
diabetes_clean <- diabetes_data %>%
  na.omit()

# Option 2: Impute missing values (if needed)
# library(mice)
# diabetes_imp <- mice(diabetes_data, m = 5)

# Check final dataset
cat("Original rows:", nrow(diabetes_data), "\n")
cat("Clean rows:", nrow(diabetes_clean), "\n")
cat("Removed:", nrow(diabetes_data) - nrow(diabetes_clean), "\n")
```

Document your choice: Removal vs. imputation affects results

Target Distribution

```r
# Class distribution
table(diabetes_clean$diabetes)
prop.table(table(diabetes_clean$diabetes))

# Visualize
ggplot(diabetes_clean, aes(x = diabetes)) +
  geom_bar(fill = "steelblue") +
  labs(title = "Diabetes Class Distribution",
       y = "Count") +
  theme_minimal()

# Calculate imbalance ratio
class_counts <- table(diabetes_clean$diabetes)
imbalance_ratio <- max(class_counts) / min(class_counts)
cat("Imbalance Ratio:", round(imbalance_ratio, 2), "\n")
```

If Imbalanced:

- Use appropriate metrics (AUC, F1, not just accuracy)
- Consider SMOTE or class weights in caret

Train-Test Split

```r
# Stratified split to preserve class balance
set.seed(2026)
train_index <- createDataPartition(
  diabetes_clean$diabetes,
  p = 0.70,       # 70% training
  list = FALSE    # return indices
)

# Split data
train_data <- diabetes_clean[train_index, ]
test_data  <- diabetes_clean[-train_index, ]

# Verify split
cat("Train:", nrow(train_data), "Test:", nrow(test_data), "\n")
prop.table(table(train_data$diabetes))
prop.table(table(test_data$diabetes))

# Verify smoking_history distribution in splits
table(train_data$smoking_history)
table(test_data$smoking_history)
```

Stratification: Ensures similar class and category proportions

Ensemble Learning Overview

Core Idea: Combine multiple weak learners to create a strong learner

Two Main Approaches:

- **Bagging (Bootstrap Aggregating):**
  - Train models on bootstrap samples
  - Average predictions (regression) or vote (classification)
  - Reduces variance
  - Example: Random Forest
- **Boosting:**
  - Train models sequentially, focusing on errors
  - Weighted combination of predictions
  - Reduces bias
  - Example: XGBoost, AdaBoost

Ensembles often outperform single models

Bagging Theory

Algorithm:

1. Draw B bootstrap samples from training data
2. Train a model $f_b(x)$ on each bootstrap sample
3. Aggregate predictions:
   - Classification: $\hat{y} = \text{majority vote}\{f_1(x), \ldots, f_B(x)\}$
   - Regression: $\hat{y} = \frac{1}{B}\sum_{b=1}^{B} f_b(x)$

Why It Works:

- Reduces variance by averaging
- Bootstrap sampling introduces diversity
- Works best with unstable models (high variance)

Limitation:

- Trees in bagging are correlated
- Feature randomness helps (leads to Random Forest)

Random Forest Algorithm

Extension of Bagging: Add feature randomness at each split

Algorithm:

1. For $b = 1$ to B:
   1. Draw bootstrap sample of training data
   2. Grow tree T_b:
      - At each split, consider only m_{try} random features
      - Split on best feature among the m_{try}
      - Grow tree to maximum depth (no pruning)
2. Aggregate predictions across all B trees

Key Parameters:

- `ntree`: Number of trees (default: 500)
- `mtry`: Features considered at each split
- `nodesize`: Minimum samples per leaf

Why Feature Randomness Helps

Problem with Bagging: Trees are correlated because:

- Same strong predictors dominate splits
- Similar tree structures across bootstrap samples

Solution: Random feature selection at each split:

- Forces trees to consider different features
- Reduces correlation among trees
- Further reduces ensemble variance

Default mtry:

- Classification: $\sqrt{p}$ (where p = total features)
- Regression: $p/3$

Tuning mtry can improve performance

Random Forest Hyperparameters

Key Parameters to Tune:

Parameter	Description	Typical Range
`mtry`	Features per split	1 to p
`ntree`	Number of trees	100 to 2000
`nodesize`	Min samples per leaf	1 to 10
`maxdepth`	Max tree depth	5 to 30 (optional)

Tuning Priority:

1. `mtry` (most important)
2. `nodesize` (second)
3. `ntree` (increase until stable)

Start with defaults, then tune mtry

Random Forest Specification in caret

```r
# Random Forest specification
rf_spec <- list(
  method = "rf",                        # Random Forest
  trControl = trainControl(
    method = "cv",                      # 10-fold CV
    number = 10,
    savePredictions = "final",
    classProbs = TRUE,                  # For ROC/AUC
    summaryFunction = twoClassSummary
  ),
  tuneGrid = expand.grid(
    mtry = c(2, 4, 6, 8)               # Tune mtry
  ),
  ntree = 500,                          # Number of trees
  nodesize = 1,                         # Min samples per leaf
  metric = "ROC"                        # Optimize for AUC
)
```

caret handles factor variables automatically

Fitting Random Forest

```r
# Fit Random Forest using caret
rf_model <- train(
  diabetes ~ age + hypertension + heart_disease +
             smoking_history + bmi + HbA1c_level +
             blood_glucose_level,
  data = train_data,
  method = "rf",
  trControl = trainControl(
    method = "cv",
    number = 10,
    savePredictions = "final",
    classProbs = TRUE,
    summaryFunction = twoClassSummary
  ),
  tuneGrid = expand.grid(mtry = c(2, 4, 6, 8)),
  ntree = 500,
  metric = "ROC"
)

# View results
print(rf_model)
plot(rf_model)  # ROC vs mtry
```

Extracting Best Parameters

```r
# View tuning results
rf_model$results

# Extract best mtry
best_mtry <- rf_model$bestTune$mtry
cat("Best mtry:", best_mtry, "\n")

# Best ROC AUC from CV
best_roc <- max(rf_model$results$ROC)
cat("Best CV ROC:", round(best_roc, 3), "\n")

# Out-of-Bag (OOB) error estimate
cat("OOB Error Estimate:",
    round(rf_model$finalModel$err.rate[nrow(rf_model$finalModel$err.rate), "OOB"],
    3), "\n")
```

OOB Error:

- Uses samples not in bootstrap sample for validation
- Provides internal validation without separate test set
- Useful for model selection during training

Predictions from Random Forest

```r
# Predict on test set
rf_pred <- predict(rf_model, newdata = test_data)
rf_probs <- predict(rf_model, newdata = test_data, type = "prob")

# Create results dataframe
rf_results <- data.frame(
  Observed = test_data$diabetes,
  Predicted = rf_pred,
  Prob_No = rf_probs$No,
  Prob_Yes = rf_probs$Yes
)

# First 10 predictions
head(rf_results, 10)

# Confusion matrix
rf_cm <- confusionMatrix(rf_pred,
                         test_data$diabetes,
                         positive = "Yes")
print(rf_cm)
```

Type = "prob" returns class probabilities

Random Forest Performance

```r
# ROC curve
rf_roc <- roc(test_data$diabetes, rf_probs$Yes,
              levels = c("No", "Yes"), direction = "<")
plot(rf_roc, col = "darkgreen", lwd = 2,
     main = "ROC Curve: Random Forest")
abline(a = 0, b = 1, lty = 2)
cat("RF AUC:", round(auc(rf_roc), 3), "\n")

# Key metrics
cat("\nKey Metrics:\n")
cat("Accuracy:", round(rf_cm$overall["Accuracy"], 3), "\n")
cat("Sensitivity:", round(rf_cm$byClass["Sensitivity"], 3), "\n")
cat("Specificity:", round(rf_cm$byClass["Specificity"], 3), "\n")
cat("Balanced Accuracy:", round(rf_cm$byClass["Balanced Accuracy"], 3), "\n")
```

Compare with baseline logistic regression

Feature Importance (Random Forest)

```r
# Extract variable importance
rf_importance <- varImp(rf_model, scale = TRUE)
print(rf_importance)

# Plot importance
plot(rf_importance,
     main = "Variable Importance: Random Forest",
     col = "darkgreen")

# Manual extraction for custom plotting
imp_df <- as.data.frame(rf_importance$importance)
imp_df$Variable <- rownames(imp_df)
imp_df <- imp_df %>% arrange(desc(Overall))

# Top 10 predictors
ggplot(head(imp_df, 10),
       aes(x = reorder(Variable, Overall), y = Overall)) +
  geom_col(fill = "darkgreen") +
  coord_flip() +
  labs(title = "Top Predictors of Diabetes (RF)",
       x = "Variable", y = "Importance (scaled)")
```

Interpretation:

Partial Dependence Plots (Optional)

```r
# Install if needed: install.packages("pdp")
library(pdp)

# Partial dependence for top predictor
pdp_bmi <- partial(rf_model$finalModel,
                   pred.var = "bmi",
                   train = train_data[, -which(names(train_data) == "diabetes")])

plot(pdp_bmi,
     main = "Partial Dependence: BMI",
     xlab = "BMI", ylab = "Predicted Probability")

# For categorical: smoking_history
pdp_smoke <- partial(rf_model$finalModel,
                     pred.var = "smoking_history",
                     train = train_data[, -which(names(train_data) == "diabetes")])
plot(pdp_smoke)
```

PDPs show marginal effect of a feature on prediction

Random Forest Strengths

Advantages:

- High accuracy with minimal tuning
- Handles non-linearities and interactions automatically
- Robust to outliers and irrelevant features
- Provides feature importance measures
- Works well with mixed data types (numeric + categorical)
- Built-in OOB error estimation
- Less prone to overfitting than single trees

Healthcare Applications:

- Risk prediction with many potential predictors
- Feature selection for clinical decision support
- Handling missing data via surrogate splits

Good default choice for many classification problems

Random Forest Limitations

Limitations:

- Less interpretable than logistic regression
- Computationally intensive for large datasets
- Predictions are not easily explained to clinicians
- Can be biased towards features with many categories
- Extrapolation beyond training data range is unreliable

Best Practices:

- Always scale numeric features (though RF is less sensitive)
- Use OOB error or cross-validation for tuning
- Report feature importance with caution (correlated features)
- Consider SHAP values for local interpretability

Balance accuracy with interpretability needs

Out-of-Bag Error

```r
# Extract OOB error rate
oob_error <- rf_model$finalModel$err.rate
plot(oob_error[, "OOB"], type = "l",
     xlab = "Number of Trees", ylab = "OOB Error",
     main = "OOB Error vs Number of Trees")
abline(h = oob_error[nrow(oob_error), "OOB"],
       col = "red", lty = 2)

# Final OOB error
final_oob <- oob_error[nrow(oob_error), "OOB"]
cat("Final OOB Error:", round(final_oob, 3), "\n")
```

OOB Error Interpretation:

- Estimated using samples not in bootstrap sample
- Unbiased estimate of test error
- Useful for monitoring convergence (ntree)
- No need for separate validation set during training

OOB error should be close to test error

Tuning mtry

```r
# View tuning results
rf_model$results %>%
  select(mtry, ROC, Accuracy) %>%
  arrange(desc(ROC))

# Plot ROC vs mtry
ggplot(rf_model$results, aes(x = mtry, y = ROC)) +
  geom_point(size = 3) +
  geom_line() +
  labs(title = "ROC AUC vs mtry",
       x = "mtry (features per split)",
       y = "ROC AUC (10-fold CV)") +
  theme_minimal()

# Optimal mtry
optimal_mtry <- rf_model$bestTune$mtry
cat("Optimal mtry:", optimal_mtry, "\n")
```

mtry Selection:

- Too small: High variance, underfitting
- Too large: High correlation, less diversity
- Optimal: Balance between bias and variance

Increasing ntree

```r
# Check if more trees improve performance
ntree_values <- c(100, 200, 500, 1000)
rf_ntree <- train(
  diabetes ~ .,
  data = train_data,
  method = "rf",
  trControl = trainControl(method = "cv", number = 5),
  tuneGrid = data.frame(mtry = best_mtry),
  ntree = ntree_values
)

# Plot accuracy vs ntree
plot(rf_ntree, main = "Accuracy vs Number of Trees")

# Extract best ntree
best_ntree <- rf_ntree$bestTune$ntree
cat("Best ntree:", best_ntree, "\n")
```

Diminishing returns: More trees = more stable but slower

Random Forest Summary

Key Takeaways:

- Ensemble of decorrelated decision trees
- Bagging + feature randomness reduces variance
- caret automates tuning and validation
- mtry is the most important hyperparameter
- Provides built-in feature importance and OOB error

Performance Summary:

- Accuracy: XX% (typically higher than logistic)
- AUC: 0.XX (strong discrimination)
- Training time: Moderate (parallelizable)
- Interpretability: Moderate (feature importance available)

Next: XGBoost for potentially better performance with boosting

When to Use Random Forest

Use Random Forest When:

- You need high accuracy with minimal tuning
- Dataset has many features (feature selection helpful)
- Non-linear relationships and interactions expected
- Interpretability is secondary to performance
- Dataset has mixed data types (numeric + categorical)
- You want built-in feature importance

Avoid Random Forest When:

- Interpretability is critical (use logistic regression)
- Dataset is very small (¡ 100 samples)
- You need probability calibration (use Platt scaling)
- Computational resources are very limited

Random Forest is a strong baseline for many problems

Transition to XGBoost

Random Forest Recap:

- Bagging: Parallel training, reduces variance
- Feature randomness: Decorrelates trees
- Good performance, moderate interpretability

XGBoost Preview:

- Boosting: Sequential training, reduces bias
- Gradient descent on loss function
- Regularization prevents overfitting
- Often state-of-the-art performance

Both are powerful; choice depends on data and goals

Boosting Theory Overview

Core Idea: Train models sequentially, each correcting errors of previous

Algorithm (General Boosting):

1. Initialize model $F_0(x)$ (e.g., mean of target)
2. For $m = 1$ to M:
   1. Compute pseudo-residuals: $r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]$
   2. Fit weak learner $h_m(x)$ to residuals r_{im}
   3. Update: $F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$
3. Output final model $F_M(x)$

Key Parameter: ν (learning rate) controls contribution of each tree

Boosting reduces bias; careful tuning needed to avoid overfitting

Gradient Boosting

Gradient Boosting: Use gradient descent to minimize loss function
Loss Functions:

- Classification: Log-loss (cross-entropy)

$$L(y, p) = -[y \log(p) + (1 - y) \log(1 - p)]$$

- Regression: Squared error, absolute error, etc.

Weak Learners: Typically shallow decision trees (depth 3-8)
Advantages over AdaBoost:

- More flexible loss functions
- Better handling of outliers
- Gradient-based optimization is more stable

XGBoost is an optimized implementation of gradient boosting

XGBoost Advantages

XGBoost (Extreme Gradient Boosting):

Key Improvements:

- **Regularization:** L1 (Lasso) and L2 (Ridge) penalties on tree weights
- **Second-order gradients:** Uses Hessian for faster convergence
- **Sparsity-aware:** Handles missing values automatically
- **Parallel processing:** Tree construction is parallelized
- **Cache optimization:** Efficient memory usage
- **Built-in cross-validation:** Early stopping to prevent overfitting

Objective Function (Regularized):

$$\mathcal{L} = \sum_{i=1}^{n} L(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

where $\Omega(f) = \gamma T + \frac{1}{2}\lambda \|w\|^2$ penalizes tree complexity

XGBoost often achieves state-of-the-art performance

XGBoost Hyperparameters

Key Parameters to Tune:

Parameter	Description	Typical Range
`eta` (learning rate)	Step size shrinkage	0.01 to 0.3
`max_depth`	Max tree depth	3 to 10
`min_child_weight`	Min sum of weights in child	1 to 10
`subsample`	Row subsampling ratio	0.5 to 1.0
`colsample_bytree`	Column subsampling ratio	0.5 to 1.0
`gamma`	Min loss reduction for split	0 to 5
`lambda`	L2 regularization	0 to 10
`alpha`	L1 regularization	0 to 10

Tuning Strategy:

1. Start with default values
2. Tune eta and max_depth first
3. Then tune subsample and colsample_bytree
4. Finally, add regularization (lambda, alpha)

XGBoost Specification in caret

```r
# XGBoost specification
xgb_spec <- list(
  method = "xgbTree",                # XGBoost in caret
  trControl = trainControl(
    method = "cv",
    number = 10,
    savePredictions = "final",
    classProbs = TRUE,
    summaryFunction = twoClassSummary
  ),
  tuneGrid = expand.grid(
    nrounds = c(100, 200),           # Number of boosting rounds
    max_depth = c(3, 5, 7),          # Tree depth
    eta = c(0.05, 0.1, 0.3),         # Learning rate
    gamma = 0,                       # Min loss reduction
    colsample_bytree = 0.8,          # Column subsampling
    min_child_weight = 1,            # Min child weight
    subsample = 0.8                  # Row subsampling
  ),
  metric = "ROC"
)
```

Start with small grid, expand if needed

Preparing Data for XGBoost

```r
# XGBoost requires numeric matrix input
# caret handles this automatically, but good to know:

# Convert factors to dummy variables manually (optional)
train_matrix <- model.matrix(diabetes ~ . - 1, data = train_data)
test_matrix <- model.matrix(diabetes ~ . - 1, data = test_data)

# Convert target to numeric (0/1)
train_label <- ifelse(train_data$diabetes == "Yes", 1, 0)
test_label <- ifelse(test_data$diabetes == "Yes", 1, 0)

# Create DMatrix (XGBoost's internal format)
library(xgboost)
dtrain <- xgb.DMatrix(data = train_matrix, label = train_label)
dtest <- xgb.DMatrix(data = test_matrix, label = test_label)

# caret handles all this internally, so you can use formula interface
```

caret simplifies XGBoost usage with formula interface

Fitting XGBoost

```r
# Fit XGBoost using caret
xgb_model <- train(
  diabetes ~ age + hypertension + heart_disease +
             smoking_history + bmi + HbA1c_level +
             blood_glucose_level,
  data = train_data,
  method = "xgbTree",
  trControl = trainControl(
    method = "cv",
    number = 10,
    savePredictions = "final",
    classProbs = TRUE,
    summaryFunction = twoClassSummary
  ),
  tuneGrid = expand.grid(
    nrounds = c(100, 200),
    max_depth = c(3, 5, 7),
    eta = c(0.05, 0.1),
    gamma = 0,
    colsample_bytree = 0.8,
    min_child_weight = 1,
    subsample = 0.8
  ),
```

Extracting Best Parameters

```r
# View tuning results
xgb_model$results %>%
  select(nrounds, max_depth, eta, ROC) %>%
  arrange(desc(ROC)) %>%
  head(10)

# Extract best parameters
best_params <- xgb_model$bestTune
cat("Best Parameters:\n")
print(best_params)

# Best ROC AUC from CV
best_roc <- max(xgb_model$results$ROC)
cat("Best CV ROC:", round(best_roc, 3), "\n")

# Number of trees used
cat("Number of boosting rounds:", best_params$nrounds, "\n")
```

Interpretation:

- eta: Smaller = more conservative, needs more rounds
- max_depth: Larger = more complex trees, risk of overfitting
- nrounds: More rounds = better fit, but watch for overfitting

Predictions from XGBoost

```r
# Predict on test set
xgb_pred <- predict(xgb_model, newdata = test_data)
xgb_probs <- predict(xgb_model, newdata = test_data, type = "prob")

# Create results dataframe
xgb_results <- data.frame(
  Observed = test_data$diabetes,
  Predicted = xgb_pred,
  Prob_No = xgb_probs$No,
  Prob_Yes = xgb_probs$Yes
)

# First 10 predictions
head(xgb_results, 10)

# Confusion matrix
xgb_cm <- confusionMatrix(xgb_pred,
                          test_data$diabetes,
                          positive = "Yes")
print(xgb_cm)
```

Same prediction interface as other caret models

XGBoost Performance

```r
# ROC curve
xgb_roc <- roc(test_data$diabetes, xgb_probs$Yes,
               levels = c("No", "Yes"), direction = "<")
plot(xgb_roc, col = "darkred", lwd = 2,
     main = "ROC Curve: XGBoost")
abline(a = 0, b = 1, lty = 2)
cat("XGB AUC:", round(auc(xgb_roc), 3), "\n")

# Key metrics
cat("\nKey Metrics:\n")
cat("Accuracy:", round(xgb_cm$overall["Accuracy"], 3), "\n")
cat("Sensitivity:", round(xgb_cm$byClass["Sensitivity"], 3), "\n")
cat("Specificity:", round(xgb_cm$byClass["Specificity"], 3), "\n")
cat("Balanced Accuracy:", round(xgb_cm$byClass["Balanced Accuracy"], 3), "\n")
```

Compare with Random Forest and logistic regression

Feature Importance (XGBoost)

```r
# Extract variable importance
xgb_importance <- varImp(xgb_model, scale = TRUE)
print(xgb_importance)

# Plot importance
plot(xgb_importance,
     main = "Variable Importance: XGBoost",
     col = "darkred")

# Manual extraction for custom plotting
imp_df <- as.data.frame(xgb_importance$importance)
imp_df$Variable <- rownames(imp_df)
imp_df <- imp_df %>% arrange(desc(Overall))

# Top 10 predictors
ggplot(head(imp_df, 10),
       aes(x = reorder(Variable, Overall), y = Overall)) +
  geom_col(fill = "darkred") +
  coord_flip() +
  labs(title = "Top Predictors of Diabetes (XGBoost)",
       x = "Variable", y = "Importance (scaled)")
```

Importance Types in XGBoost:

SHAP Values for Interpretation (Optional)

```r
# Install if needed: install.packages("shapviz")
library(shapviz)
library(xgboost)

# Prepare data for SHAP
xgb_final <- xgb_model$finalModel
X_test <- model.matrix(diabetes ~ . - 1, data = test_data)

# Compute SHAP values
shap <- shapviz(xgb_final, X_pred = X_test, X = X_test)

# Summary plot
sv_importance(shap, show_numbers = TRUE)

# Dependence plot for top feature
sv_dependence(shap, v = "bmi")

# Force plot for individual prediction
# sv_force(shap, row_id = 1)
```

SHAP values provide local and global interpretability

Early Stopping (Advanced)

```r
# Early stopping prevents overfitting
# caret doesn't support it directly, but you can use xgboost directly:

library(xgboost)

# Prepare data
dtrain <- xgb.DMatrix(data = train_matrix, label = train_label)
dval <- xgb.DMatrix(data = test_matrix, label = test_label)

# Train with early stopping
xgb_params <- list(
  objective = "binary:logistic",
  eta = 0.1,
  max_depth = 5,
  subsample = 0.8,
  colsample_bytree = 0.8
)

xgb_early <- xgb.train(
  params = xgb_params,
  data = dtrain,
  nrounds = 1000,
  watchlist = list(val = dval),
```

XGBoost Strengths

Advantages:

- Often achieves state-of-the-art performance
- Handles missing values automatically
- Built-in regularization prevents overfitting
- Efficient implementation (fast training)
- Flexible loss functions
- Provides feature importance
- Works well with structured/tabular data

Healthcare Applications:

- High-stakes prediction where accuracy is critical
- Large datasets with many features
- When you can afford computational cost for tuning

XGBoost is a top choice for Kaggle competitions and production ML

XGBoost Limitations

Limitations:

- Many hyperparameters to tune (complex)
- Less interpretable than simpler models
- Can overfit if not properly regularized
- Requires careful cross-validation
- Training can be slow without parallelization
- Probability outputs may need calibration

Best Practices:

- Start with conservative hyperparameters
- Use cross-validation for all tuning
- Monitor training vs validation performance
- Consider probability calibration (Platt scaling)
- Use SHAP values for interpretability

More powerful but requires more expertise than Random Forest

Tuning Strategy for XGBoost

Recommended Tuning Order:

1. **Step 1:** Tune `eta` and `max_depth`
   - Start with eta = 0.1, max_depth = 5
   - Grid search small ranges first
2. **Step 2:** Tune `subsample` and `colsample_bytree`
   - Add randomness to reduce overfitting
   - Typical range: 0.6 to 1.0
3. **Step 3:** Add regularization (`lambda`, `alpha`)
   - Start with 0, increase if overfitting
   - L2 (lambda) is usually more effective than L1
4. **Step 4:** Fine-tune `nrounds` with early stopping

Tune incrementally; don't grid search all parameters at once

Parallel Processing (Optional)

```r
# Speed up training with parallel processing
library(doParallel)

# Register parallel backend
cl <- makeCluster(detectCores() - 1)  # Leave 1 core free
registerDoParallel(cl)

# Train with parallel processing
rf_model_parallel <- train(
  diabetes ~ .,
  data = train_data,
  method = "rf",
  trControl = trainControl(
    method = "cv",
    number = 10,
    allowParallel = TRUE  # Enable parallel
  ),
  tuneGrid = expand.grid(mtry = c(2, 4, 6, 8)),
  ntree = 500
)

# Stop cluster
stopCluster(cl)
```

Model Comparison Setup

```r
# Collect predictions from all models
all_predictions <- data.frame(
  Observed = test_data$diabetes,
  Logistic = glm_pred_class,      # From baseline
  RF = rf_pred,                   # Random Forest
  XGB = xgb_pred                  # XGBoost
)

# Collect probabilities for ROC comparison
all_probs <- data.frame(
  Observed = test_data$diabetes,
  Logistic = glm_probs,           # From baseline
  RF = rf_probs$Yes,
  XGB = xgb_probs$Yes
)
```

Next: Compute and compare metrics across all models

Comparing ROC Curves

```r
# Plot all ROC curves together
plot(glm_roc, col = "steelblue", lwd = 2,
     main = "ROC Comparison: All Models")
lines(rf_roc, col = "darkgreen", lwd = 2)
lines(xgb_roc, col = "darkred", lwd = 2)
abline(a = 0, b = 1, lty = 2, col = "gray")
legend("bottomright",
       legend = c("Logistic", "Random Forest", "XGBoost"),
       col = c("steelblue", "darkgreen", "darkred"),
       lwd = 2)

# Compare AUC values
auc_values <- data.frame(
  Model = c("Logistic", "Random Forest", "XGBoost"),
  AUC = c(as.numeric(glm_auc),
          as.numeric(auc(rf_roc)),
          as.numeric(auc(xgb_roc)))
)
print(auc_values)
```

Visual comparison helps identify best-performing model

Comparing Metrics Table

```r
# Create comparison table
comparison_table <- data.frame(
  Model = c("Logistic", "Random Forest", "XGBoost"),
  Accuracy = c(
    glm_cm$overall["Accuracy"],
    rf_cm$overall["Accuracy"],
    xgb_cm$overall["Accuracy"]
  ),
  Sensitivity = c(
    glm_cm$byClass["Sensitivity"],
    rf_cm$byClass["Sensitivity"],
    xgb_cm$byClass["Sensitivity"]
  ),
  Specificity = c(
    glm_cm$byClass["Specificity"],
    rf_cm$byClass["Specificity"],
    xgb_cm$byClass["Specificity"]
  ),
  AUC = c(
    as.numeric(glm_auc),
    as.numeric(auc(rf_roc)),
    as.numeric(auc(xgb_roc))
  )
```

Model Selection Recommendations

For Diabetes Prediction:

Priority	Recommended Model	Reason
Interpretability	Logistic Regression	Coefficients, odds ratios
Accuracy	XGBoost	Often highest AUC
Speed	Random Forest	Fast training, good accuracy
Small Data	Random Forest	Less prone to overfitting
Large Data	XGBoost	Scales well, high performance
Feature Selection	Random Forest	Built-in importance

Our Recommendation:

- Start with Random Forest for balance of performance and ease of use
- If accuracy is critical and resources allow, try XGBoost
- Always validate with cross-validation and hold-out test set
- Consider model interpretability requirements for healthcare

No single best model; choose based on context and constraints

Practical Implementation Checklist

Before Modeling:

1. Define target variable clearly (avoid data leakage)
2. Handle missing values appropriately
3. Convert categorical variables to factors
4. Check category distributions (e.g., smoking_history)
5. Split data before any preprocessing

During Modeling:

1. Start with simple baseline (logistic regression)
2. Use cross-validation for reliable performance estimates
3. Tune hyperparameters on validation set only
4. Monitor training time and computational resources
5. Save preprocessing parameters for deployment

After Modeling:

1. Evaluate on held-out test set (never used in training/tuning)
2. Report multiple metrics (not just accuracy)
3. Document model limitations and assumptions

Healthcare Application Considerations

Ethical Considerations:

- **Bias:** Ensure training data represents all demographics
- **Privacy:** Protect patient health information
- **Transparency:** Explain model predictions to clinicians
- **Accountability:** Human-in-the-loop for medical decisions

Deployment Considerations:

- **Latency:** XGBoost can be slower than Random Forest
- **Scalability:** Both scale well with parallel processing
- **Monitoring:** Track model drift over time
- **Updates:** Retrain with new data periodically

Warning: ML models assist, not replace, clinical judgment

Course Summary

What We Covered:

1. Data preparation with multi-level categorical variables
2. Random Forest: Bagging + feature randomness
3. XGBoost: Gradient boosting with regularization
4. Hyperparameter tuning with cross-validation
5. Feature importance interpretation
6. Model comparison and selection
7. Healthcare application considerations

Key Skills Developed:

- R programming for ensemble methods
- caret workflow for model training and tuning
- Performance metric interpretation
- Model selection for specific applications
- Ethical considerations in healthcare ML

Next Steps: Practice with your own datasets, explore deep learning

Best Recommendations

Practice Recommendations:

- Try different hyperparameter grids for each model
- Experiment with alternative preprocessing (e.g., PCA, feature selection)
- Compare with tidymodels workflow for modern alternative
- Apply to your own health datasets with appropriate ethics approval

Next Session: Model Deployment and Monitoring
Topics: Saving models, API deployment, drift detection

"The best model is the one that solves your problem, not the most complex one."

Thank You!!!

Questions?

Instructor: Lumumba W.V.
Email: cdam@chuka.ac.ke — lumumbavictor172@gmail.com
Phone: +254 706 038 599
Department: Center for Data Analytics and Modeling
Institution: Chuka University, Kenya

Resources:

- Course Materials: `https://cdam.chuka.ac.ke`
- R Code Repository: `https://github.com/cdam-chuka/r-sessions`
- caret Documentation: `https://topepo.github.io/caret/`
- XGBoost Documentation: `https://xgboost.readthedocs.io`
- Diabetes Dataset: Diabetes.csv (this session)